

DE LA PLUME ET DU TEXTE

Érasme, saisi par Holbein la plume à la main, corrige son texte mot à mot dans la marge : VS Code offre au code ce que l'écritoire offrait à l'humaniste, un espace pour écrire, corriger et reprendre, phrase après phrase.



FIG. VII. HANS HOLBEIN LE JEUNE, « ÉRASME ÉCRIVANT », 1523

Tutoriel complet de Visual Studio Code : installation, interface, extensions Python/Jupyter/débogueur, Git intégré et automatisation, en trois parcours de niveau.

SOMMAIRE

1. L'EXTENSION PYTHON EN PROFONDEUR

2. L'EXTENSION JUPYTER : NOTEBOOKS DANS VS CODE

3. LE DÉBOGUEUR : POINTS D'ARRÊT ET EXÉCUTION PAS À PAS

4. GIT INTÉGRÉ : SUIVRE L'ÉVOLUTION D'UN PROJET

5. SNIPPETS, RACCOURCIS ET PALETTE DE COMMANDES

6. PLUSIEURS LANGAGES DANS UN MÊME PROJET

7. ESPACES DE TRAVAIL MULTI-DOSSIERS ET PARAMÈTRES PAR PROJET

Visual Studio Code (VS Code) est un éditeur de code gratuit, open-source et multiplateforme, développé par Microsoft. Il n'est pas un logiciel de géomatique en soi — contrairement à QGIS ou TerrSet — mais il est devenu l'environnement de travail standard pour écrire, déboguer et versionner les scripts Python (GeoPandas, PyQGIS, rasterio...) et R qui automatisent les traitements présentés dans les modules Le Compas et travaux pratiques. Ce module en fait un tutoriel complet, en trois pistes (choisis la tienne dans le filtre « Afficher ») : chacune se lit seule, du début à la fin.

1. L'EXTENSION PYTHON EN PROFONDEUR

Un script géomatique dépend presque toujours de bibliothèques externes (GeoPandas, rasterio, PyQGIS) installées dans un environnement Python précis — souvent un environnement virtuel dédié au projet, distinct de l'installation Python du système. VS Code doit savoir lequel utiliser.

- Palette de commandes (Ctrl+Shift+P) > taper « Python: Select Interpreter » : liste tous les environnements Python détectés sur la machine (installation système, environnements virtuels venv, environnements conda)
- L'interpréteur sélectionné s'affiche dans la barre d'état en bas ; c'est lui qui est utilisé par le bouton Run, le terminal intégré ouvert ensuite, et l'autocomplétion
- Un environnement virtuel mal sélectionné explique la majorité des erreurs « ModuleNotFoundError » alors que le module est pourtant installé — installé, mais dans un autre environnement que celui actif

ATTENTION

UNE ERREUR MODULENOTFOUNDERROR N'EST PAS TOUJOURS UN MODULE MANQUANT

`import geopandas` qui échoue alors que `pip install geopandas` a réussi vient presque toujours d'un interpréteur mal sélectionné : le module est installé dans un environnement, VS Code exécute avec un autre. Vérifier l'interpréteur actif (barre d'état, ou Python: Select Interpreter) est le premier réflexe avant de réinstaller quoi que ce soit.

- Linting : détection d'erreurs et de style avant même l'exécution, soulignées directement dans l'éditeur (extension Ruff ou Pylance, incluse avec l'extension Python)
- Formatage automatique : réécrit le code selon une convention de style cohérente (espaces, longueur de ligne) sans en changer le comportement — Black ou Ruff formatter, déclenché manuellement (Shift+Alt+F) ou automatiquement à l'enregistrement

PYLANCE

- Moteur d'analyse installé avec l'extension Python (Microsoft)
- Autocomplétion, vérification de types, navigation (aller à la définition)

- Souligne les erreurs probables avant toute exécution

RUFF

- Linter et formateur très rapide (écrit en Rust), remplace souvent Flake8 + Black à lui seul
- Signale les imports inutilisés, variables non utilisées, style non conforme (PEP 8)
- Peut reformater tout le fichier en un raccourci ou à chaque enregistrement

ACTIVER LE FORMATAGE AUTOMATIQUE À L'ENREGISTREMENT POUR PYTHON

```
"[python]": { "editor.formatOnSave": true,  
"editor.defaultFormatter": "charliermarsh.ruff" }
```

À ajouter dans settings.json (Ctrl+Shift+P > Preferences: Open User Settings (JSON)) : reformate automatiquement le fichier Python à chaque Ctrl+S, avec Ruff comme formateur.

SUPÉRIEUR

2. L'EXTENSION JUPYTER : NOTEBOOKS DANS VS CODE

Un notebook Jupyter (fichier .ipynb) mélange cellules de code exécutables et cellules de texte, avec un résultat affiché immédiatement sous chaque cellule — pratique pour explorer un jeu de données pas à pas, sans tout relancer depuis le début à chaque modification.

- Installer l'extension Jupyter (Microsoft) depuis le Marketplace, en complément de l'extension Python
- Créer un fichier .ipynb (ou Ctrl+Shift+P > Jupyter: Create New Jupyter Notebook) : chaque cellule s'exécute indépendamment avec Shift+Entrée
- Le résultat d'une cellule (texte, tableau, graphique matplotlib) s'affiche directement sous elle, sans quitter l'éditeur
- Le panneau Variables (bouton en haut du notebook) liste toutes les variables actives en mémoire — leur type, leur valeur, très utile pour inspecter un GeoDataFrame intermédiaire sans taper `gdf.head()` à chaque fois

EXEMPLE

EXEMPLE CONCRET : INSPECTER UN GEODATAFRAME CELLULE PAR CELLULE

Une première cellule charge `gdf = gpd.read_file("parcelles.gpkg")` et l'exécute (Shift+Entrée) ; une deuxième cellule, exécutée seule sans relancer la première, affiche `gdf.plot()` ou `gdf.head()` pour vérifier le résultat immédiatement — puis, si une colonne manque, on modifie uniquement la cellule concernée et on la relance, sans recharger tout le fichier depuis le début.

REMARQUE

NOTEBOOK OU SCRIPT : QUAND CHOISIR LEQUEL

Un notebook convient à l'exploration (tester une transformation, visualiser un résultat intermédiaire, comparer plusieurs approches côte à côte). Un script `.py` classique convient à un traitement destiné à être réexécuté tel quel, automatisé ou intégré à une chaîne de production — un notebook exécuté dans le désordre (cellule 5 avant cellule 3) peut donner un résultat qui dépend de l'ordre d'exécution réel, pas de l'ordre affiché à l'écran.

SUPÉRIEUR

3. LE DÉBOGUEUR : POINTS D'ARRÊT ET EXÉCUTION PAS À PAS

Multiplier les `print()` pour comprendre pourquoi un script échoue devient vite ingérable dès que le script dépasse quelques dizaines de lignes : il faut deviner où insérer chaque `print()`, puis les retirer un par un une fois le problème résolu. Le débogueur intégré remplace cette méthode par une inspection directe, à l'endroit exact où le calcul dérape.

- Cliquer dans la marge à gauche d'une ligne (juste à gauche du numéro de ligne) pour poser un point d'arrêt (un point rouge apparaît)
- F5 (ou l'icône Exécution et débogage dans la barre d'activité) lance le script en mode débogage : l'exécution se suspend automatiquement au premier point d'arrêt rencontré

- Une fois suspendu : le panneau Variables affiche le contenu exact de chaque variable à cet instant précis (un GeoDataFrame, sa géométrie, son CRS)
- F10 (Step Over) exécute la ligne courante sans entrer dans une fonction appelée ; F11 (Step Into) entre dans la fonction appelée pour l'inspecter ligne par ligne ; Shift+F11 (Step Out) ressort de la fonction en cours jusqu'à l'appelant
- F5 depuis un point d'arrêt reprend l'exécution normale jusqu'au point d'arrêt suivant (ou la fin du script)

TOUCHE	ACTION EN MODE DÉBOGAGE
F5	Lancer le débogage, ou reprendre jusqu'au point d'arrêt suivant
F9	Basculer un point d'arrêt sur la ligne courante (équivalent au clic dans la marge)
F10	Step Over : exécuter la ligne sans entrer dans une fonction appelée
F11	Step Into : entrer dans la fonction appelée
Shift+F11	Step Out : ressortir de la fonction courante
Shift+F5	Arrêter le débogage

REMARQUE

UN POINT D'ARRÊT VAUT MIEUX QU'UN PRINT()

Poser un point d'arrêt suspend le script à l'endroit exact où quelque chose se passe mal et permet d'inspecter le contenu réel de chaque variable — au lieu de deviner où ça casse en semant des `print()` qu'il faut ensuite retirer un par un. C'est la différence entre chercher une fuite en démontant tout le mur et poser un capteur précisément là où l'humidité apparaît.

Le fichier `.vscode/launch.json` (créé automatiquement au premier F5, ou via `Ctrl+Shift+P > Debug: Add Configuration`) définit comment le débogueur doit lancer le script : quel interpréteur, quels arguments, quel dossier de travail.

EXTRAIT DE LAUNCH.JSON POUR UN SCRIPT PYTHON

```
{ "name": "Python: Fichier actuel", "type": "debugpy",  
  "request": "launch", "program": "${file}", "console":  
  "integratedTerminal" }
```

`${file}` désigne le fichier actuellement ouvert dans l'éditeur : cette configuration permet de déboguer n'importe quel script Python ouvert, sans en fixer le nom à l'avance.

SUPÉRIEUR

4. GIT INTÉGRÉ : SUIVRE L'ÉVOLUTION D'UN PROJET

Git suit l'historique des modifications d'un projet, fichier par fichier, comme un plan de carte qu'on versionnerait à chaque étape plutôt que d'écraser la version précédente. VS Code intègre Git nativement, sans extension supplémentaire.

- Ctrl+Shift+G ouvre le panneau Source Control (Contrôle de source), qui liste tous les fichiers modifiés depuis le dernier commit
- Cliquer sur le + à côté d'un fichier pour le mettre en zone de préparation (stage) — ou stage tout d'un coup
- Taper un message de commit dans le champ en haut du panneau, puis Ctrl+Entrée (ou le bouton Commit) pour valider ces modifications dans l'historique
- Le bouton Sync / Push envoie les commits locaux vers un dépôt distant (GitHub, GitLab...)
- Cliquer sur un fichier modifié dans le panneau ouvre une vue de comparaison (diff) côte à côte ou en ligne, avec les suppressions en rouge et les ajouts en vert

ACTION	OÙ, DANS VS CODE
Voir les fichiers modifiés	Panneau Source Control (Ctrl+Shift+G)
Stage un fichier	Icône + à côté du fichier, dans le panneau Source Control
Commit	Champ de message en haut du panneau, puis Ctrl+Entrée
Push / Pull	Bouton Sync (flèches circulaires) en bas de la fenêtre, ou menu ... du panneau
Comparer une version antérieure	Clic droit sur un fichier > Open Timeline, ou l'extension GitLens

EXEMPLE

RÉSOUTRE UN CONFLIT DE FUSION DIRECTEMENT DANS L'ÉDITEUR

Lors d'un conflit (deux modifications incompatibles sur la même ligne d'un fichier), VS Code affiche directement dans l'éditeur les deux versions en conflit, encadrées de marqueurs <<<<<<, ===== et >>>>>>, avec des boutons cliquables « Accept Current Change », « Accept Incoming Change » ou « Accept Both Changes » au-dessus de chaque bloc — pas besoin de retourner en ligne de commande pour éditer les marqueurs à la main.

SUPÉRIEUR

5. SNIPPETS, RACCOURCIS ET PALETTE DE COMMANDES

- Ctrl+Shift+P (Cmd+Shift+P sur macOS) ouvre la palette de commandes : point d'entrée universel pour toute action de VS Code, accessible en tapant son nom plutôt qu'en cherchant un menu
- Ctrl+P ouvre rapidement un fichier par son nom, sans passer par l'explorateur
- Un snippet est un raccourci qui insère un bloc de code type (ex. taper for puis Tab insère automatiquement une boucle for avec le curseur positionné au bon endroit) ; Ctrl+Shift+P > Snippets: Configure Snippets permet d'en créer des personnalisés

- Ctrl+/ commente/décommente la ligne ou la sélection courante ; Alt+↑/↓ déplace la ligne courante vers le haut ou le bas

RACCOURCI	ACTION
Ctrl+P	Ouvrir rapidement un fichier par son nom
Ctrl+Shift+P	Palette de commandes
Ctrl+/	Commenter / décommenter la sélection
Alt+↑ / Alt+↓	Déplacer la ligne courante vers le haut / le bas
Ctrl+D	Sélectionner la prochaine occurrence du mot courant (édition multi-curseur)
F2	Renommer le symbole sous le curseur dans tout le fichier (voire tout le projet, selon le langage)
Ctrl+Espace	Déclencher manuellement l'autocomplétion

EXEMPLE DE SNIPPET PERSONNALISÉ POUR GEOPANDAS

```
"Charger un GeoDataFrame": { "prefix": "gdfread", "body":
["gdf = gpd.read_file($1)", "gdf = gdf.to_crs(epsg=2154)"] }
```

Défini dans un fichier de snippets Python (Ctrl+Shift+P > Snippets: Configure Snippets > python.json) : taper gdfread puis Tab insère les deux lignes, avec le curseur positionné sur \$1 (le chemin du fichier) prêt à être complété.

SUPÉRIEUR

6. PLUSIEURS LANGAGES DANS UN MÊME PROJET

Un projet géomatique mélange rarement un seul langage : un script Python de traitement, des requêtes SQL vers une base PostGIS, un fichier README en Markdown pour documenter la démarche. VS Code gère cette diversité par une extension dédiée à chaque langage, toutes actives simultanément dans le même projet.

- Extension Python (Microsoft) pour les scripts .py

- Extension SQLTools ou PostgreSQL pour écrire et exécuter des requêtes SQL directement dans l'éditeur, avec autocomplétion sur les noms de tables/colonnes
- Aperçu Markdown intégré nativement (Ctrl+Shift+V) pour prévisualiser un README.md tel qu'il s'affichera sur GitHub
- Chaque extension reste indépendante : ouvrir un fichier .sql ou .md n'active que l'extension correspondante, sans interférer avec celle utilisée pour les fichiers .py

TYPE DE FICHIER	EXTENSION RECOMMANDÉE	CE QU'ELLE APPORTE
.py	Python (Microsoft)	Coloration, autocomplétion, exécution, linting
.sql	SQLTools ou PostgreSQL	Autocomplétion sur les tables/colonnes, exécution de requêtes
.md	Aperçu natif (Ctrl+Shift+V)	Aucune extension requise, l'aperçu Markdown est intégré
.json / .yaml	Prise en charge native	Validation de syntaxe, pliage des sections
.R	R (REditorSupport)	Coloration, exécution, intégration avec Radian

SUPÉRIEUR

7. ESPACES DE TRAVAIL MULTI-DOSSIERS ET PARAMÈTRES PAR PROJET

Au-delà d'un simple dossier, un espace de travail (workspace) VS Code peut regrouper plusieurs dossiers distincts (par exemple un dossier de scripts et un dossier de données séparé) dans une même fenêtre, sauvegardé dans un fichier .code-workspace.

- File > Add Folder to Workspace... ajoute un second dossier à l'espace de travail courant
- File > Save Workspace As... enregistre cette configuration dans un fichier .code-workspace, réouvrable en un double-clic

- Le dossier `.vscode/` à la racine d'un projet contient des paramètres propres à ce projet uniquement (`settings.json`), qui s'appliquent seulement quand ce dossier est ouvert — sans modifier la configuration globale de VS Code

EXTRAIT DE `.VSCODE/SETTINGS.JSON` : PARAMÈTRES PROPRES À UN PROJET

```
{ "python.defaultInterpreterPath": "./venv/bin/python",  
  "editor.rulers": [88] }
```

Ce fichier, versionné avec le projet (contrairement aux paramètres utilisateur globaux), fixe l'environnement virtuel à utiliser et une règle visuelle à 88 caractères : partagé avec toute personne qui clone le projet, la configuration reste identique pour tout le monde.

- Bilan — à retenir : Python: Select Interpreter choisit l'environnement virtuel actif, source la plus fréquente d'un `ModuleNotFoundError` trompeur ; l'extension Jupyter exécute un notebook `.ipynb` cellule par cellule avec un explorateur de variables ; le débogueur (point d'arrêt, F10/F11, `launch.json`) inspecte une variable précisément là où ça casse, plutôt que par des `print()` ; le panneau Source Control (`Ctrl+Shift+G`) gère stage/commit/push et affiche les conflits de fusion directement dans l'éditeur ; la palette de commandes (`Ctrl+Shift+P`) est le point d'entrée universel ; `.vscode/settings.json` fixe des paramètres propres à un projet, versionnés avec lui.

VOIR AUSSI : PRATIQUER : LES SCRIPTS GEOPANDAS ET PYQGIS ÉDITÉS DANS VS CODE

Le module Le Compas (section 6) détaille PyQGIS et GeoPandas : les bibliothèques que cet environnement sert précisément à écrire, déboguer et versionner.

VOIR AUSSI : VOIR AUSSI : R DANS VS CODE

Le module De l'Instrument et du Nombre présente R ; la piste Master de ce module montre comment l'exécuter directement dans VS Code, aux côtés de Python.